Task: Hypervisor Extension

Two-Stage Translation and Guest Memory Access

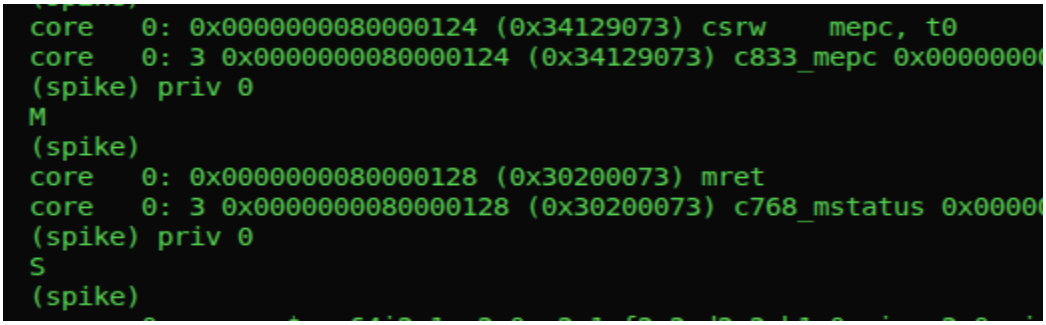
Learning Objectives:
1. Privilege hierarchy: M > HS > VS > VU
2. Two-stage address translation (vsatp + hgatp)
3. HLV/HSV instructions for guest memory access
4. Trap delegation (medeleg, hedeleg)
5. Key CSRs: vsstatus, hstatus, vsatp, hgatp

Task Description:
Implement a hypervisor managing a guest OS in VS-mode with VU-mode application.

Implementation Steps:
1. M-mode initialization:
* Setup mtvec
* Delegate exceptions via medeleg (bits 8, 10, 12, 13, 15)
* Initialize hgatp for G-stage translation
* Jump to HS-mode using mret
2. HS-mode hypervisor:
* Setup stvec
* Configure hstatus (enable virtualization)
* Setup vsatp for guest first-stage translation
* Jump to VS-mode (set hstatus.SPV=1, sstatus.SPP=1, use sret)
3. VS-mode guest OS:
* Store test value 0xDEADBEEF at known address
* Jump to VU-mode (set sstatus.SPP=0, use sret)
4. VU-mode application:
* Pass VS memory address in a0
* Execute ecall (traps to HS-mode)
5. HS-mode trap handler:
* Check scause (should be 8 for VU-mode ecall)
* Set hstatus.SPVP=0
* Use HLV.WU to read from VS memory address in a0
* Modify value (add 0x100)
* Use HSV.W to write back
* Set sepc to VS verification code
* Set sstatus.SPP=1, keep hstatus.SPV=1
* Execute sret to VS-mode6. VS-mode verification:
* Use LWU to load modified value (avoid sign extension)
* Verify: 0xDEADBEEF + 0x100 = 0xDEADBFEF
* Execute ecall to HS-mode
7. HS-mode exit:
* Check scause (should be 10 for VS-mode ecall)
* Execute ecall to M-mode
8. M-mode exit:
* Check mcause (should be 9 for HS-mode ecall)
* Skip ecall (mepc += 4)
* Write success to tohost

Submit a PDF with Screenshots that shows:

a): Confirm M→HS→VS→VU mode transitions worked



M -> HS transition

```
core   0: 0x00000000800020b4 (0x14129073) csrw    sepc, t0
core   0: 1 0x00000000800020b4 (0x14129073) c321_sepc 0x0000
(spike) priv 0
S
(spike)
core   0: 0x00000000800020b8 (0x10200073) sret
core   0: 1 0x00000000800020b8 (0x10200073) c768_mstatus 0x0
(spike) priv 0
VS
```

HS -> VS transition

```
core   0: 0x0000000080003038 (0x14129073) csrw    sepc, t0
core   0: 1 0x0000000080003038 (0x14129073) c577_vsepc 0x000000
(spike) priv 0
VS
(spike)
core   0: 0x000000008000303c (0x10200073) sret
core   0: 1 0x000000008000303c (0x10200073) c512_vsstatus 0x000
(spike) priv 0
VU
(spike)
```

VS -> VU transition

b): Verify medeleg includes both bit 8 (VU ecall) and bit 10 (VS ecall)

```
(spike) priv 0
VU
(spike)
core   0: 0x0000000080004008 (0xffc50513) addi    a0, a0, -4
core   0: 0 0x0000000080004008 (0xffc50513) x10 0x0000000080011000
(spike) priv 0
VU
(spike)
core   0: 0x000000008000400c (0x00000073) ecall
core   0: exception trap_user_ecall, epc 0x000000008000400c
(spike) priv 0
S
(spike)
core   0: >>>>  trap_handler_s
core   0: 0x0000000080004010 (0x142022f3) csrr    t0, scause
```

8 (VU ecall)

```
(spike) priv 0
VS
(spike)
core   0: 0x000000008000501c (0x02731863) bne     t1, t2, pc + 48
core   0: 1 0x000000008000501c (0x02731863)
(spike) priv 0
VS
(spike)
core   0: 0x0000000080005020 (0x00000073) ecall
core   0: exception trap_virtual_supervisor_ecall, epc 0x000000008000
(spike) priv 0
S
(spike)
core   0: >>>>  trap_handler_s
core   0: 0x0000000080004010 (0x142022f3) csrr    t0, scause
```

10 (VS ecall

c): Confirm HS trap handler checks scause=10 for VS ecalls (NOT 9)

```
(spike) priv 0
VS
(spike)
core   0: 0x0000000080005020 (0x00000073) ecall
core   0: exception trap_virtual_supervisor_ecall, epc 0x0000000080005020
(spike) priv 0
S
(spike)
core   0: >>>>  trap_handler_s
core   0: 0x0000000080004010 (0x142022f3) csrr    t0, scause
core   0: 1 0x0000000080004010 (0x142022f3) x5  0x000000000000000a
(spike)
core   0: 0x0000000080004014 (0x00a00313) li      t1, 10
core   0: 1 0x0000000080004014 (0x00a00313) x6  0x000000000000000a
(spike)
core   0: 0x0000000080004018 (0x00629663) bne     t0, t1, pc + 12
```

d): Verify VU-mode uses command codes in a1 to distinguish different ecall purposes

```
core   0: 0x000000008000400c (0x00100593) li      a1, 1
core   0: 0 0x000000008000400c (0x00100593) x11 0x0000000000000001
(spike)
core   0: 0x0000000080004010 (0x00000073) ecall
core   0: exception trap_user_ecall, epc 0x0000000080004010
```

```
core   0: 0x0000000080004034 (0x00100313) li      t1, 1
core   0: 1 0x0000000080004034 (0x00100313) x6  0x0000000000000001
(spike)
core   0: 0x0000000080004038 (0x00658463) beq     a1, t1, pc + 8
core   0: 1 0x0000000080004038 (0x00658463)
```

a1=1 user ecall

```
core   0: 0x0000000080005020 (0x00200593) li      a1, 2
core   0: 1 0x0000000080005020 (0x00200593) x11 0x0000000000000002
(spike)
core   0: 0x0000000080005024 (0x00000073) ecall
core   0: exception trap_virtual_supervisor_ecall, epc 0x0000000080005024
(spike)
core   0: >>>>  trap_handler_s
core   0: 0x0000000080004014 (0x142022f3) csrr    t0, scause
core   0: 1 0x0000000080004014 (0x142022f3) x5  0x000000000000000a
(spike)
core   0: 0x0000000080004018 (0x00a00313) li      t1, 10
core   0: 1 0x0000000080004018 (0x00a00313) x6  0x000000000000000a
(spike)
core   0: 0x000000008000401c (0x00629c63) bne     t0, t1, pc + 24
core   0: 1 0x000000008000401c (0x00629c63)
(spike)
core   0: 0x0000000080004020 (0x00200313) li      t1, 2
core   0: 1 0x0000000080004020 (0x00200313) x6  0x0000000000000002
(spike)
core   0: 0x0000000080004024 (0x00658463) beq     a1, t1, pc + 8
core   0: 1 0x0000000080004024 (0x00658463)
```

a1=2 virtual supervisor ecall

e): Verify VS-mode stored data correctly (0xDEADBEEF)

```
core   0: 0x000000008000301c (0x0062a023) sw      t1, 0(t0)
core   0: 1 0x000000008000301c (0x0062a023) mem 0x0000000080011000 0xdeadbeef
(spike) mem 0 0x0000000080011000
0x00000000deadbeef
(spike) priv 0
VS
```

f): Confirm HLV.WU (not HLV.W) read correct value from VS memory without sign extension

```
core   0: 0x0000000080004048 (0x6812c373) hlv.wu  t1, (t0)
core   0: 1 0x0000000080004048 (0x6812c373) x6  0x00000000deadbeef mem 0x0000000080011000
(spike) reg 0 t1
0x00000000deadbeef
(spike) priv 0
S
(spike)
```

g): Verify HSV.W wrote modified value (0xDEADBFEF) back to VS memory

```
core   0: 0x0000000080004068 (0x6a62c073) hsv.w   t1, (t0)
core   0: 1 0x0000000080004068 (0x6a62c073) mem 0x0000000080011000 0xdeadbfef
(spike) reg 0 t1
0x00000000deadbfef
(spike) priv 0
S
(spike)
```

h): Confirm VS-mode uses LWU (not LW) to load and verify modified value

```
core   0: 0x0000000080005008 (0x0002e303) lwu     t1, 0(t0)
core   0: 1 0x0000000080005008 (0x0002e303) x6  0x00000000deadbfef mem 0x0000000080011000
(spike) reg 0 t1
0x00000000deadbfef
(spike) priv 0
VS
(spike)
```

i): Verify HLVX.WU successfully fetched instruction from VS code region

```
core   0: 0x0000000080004088 (0x6832c373) hlvx.wu t1, (t0)
core   0: 1 0x0000000080004088 (0x6832c373) x6  0x0000000000000013 mem 0x0000000080003000
(spike) reg 0 t1
0x0000000000000013
(spike)
core   0: 0x000000008000408c (0x01300393) li      t2, 19
core   0: 1 0x000000008000408c (0x01300393) x7  0x0000000000000013
(spike)
core   0: 0x0000000080004090 (0x7a731ee3) bne     t1, t2, pc + 4028
```

j): Confirm two-stage translation occurred

```
(spike) reg 0 hgatp
0x8000000000080008
(spike) reg 0 vsatp
0x800000000008000e
(spike)
```

Both CSRs active simultaneously. vsatp (MODE=8, Sv39) controls VS-stage and hgatp (MODE=8, Sv39x4) controls G-stage. Both are non-zero during guest execution (V=1), meaning hardware applies both stages to every memory access.

Guest store succeeded. VS-mode stored 0xDEADBEEF to load_data virtual address. This address was translated first by VS-stage (vsatp page tables) to a guest physical address, then by G-stage (hgatp page tables) to a real physical address. The store appearing in memory confirms both translations completed correctly.

k) Verify proper mode transitions: M→HS→VS→VU→HS→VS→HS→M

```
(spike) priv 0
M
(spike)
core   0: 0x0000000080000128 (0x30200073) mret
core   0: 3 0x0000000080000128 (0x30200073) c768_mstatus 0x0000000a00000080
(spike) priv 0
S
(spike)
```

```
(spike) priv 0
S
(spike)
core   0: 0x00000000800020b8 (0x10200073) sret
core   0: 1 0x00000000800020b8 (0x10200073) c768_mstatus 0x0000000a000000a0 c15
(spike) priv 0
VS
(spike)
```

```
(spike) priv 0
VS
(spike)
core   0: 0x000000008000303c (0x10200073) sret
core   0: 1 0x000000008000303c (0x10200073) c512_vsstatus 0x0000000200000020
(spike) priv 0
VU
(spike)
```

```
(spike) priv 0
VU
(spike)
core   0: 0x000000008000400c (0x00000073) ecall
core   0: exception trap_user_ecall, epc 0x000000008000400c
(spike) priv 0
S
(spike)
```

```
(spike) priv 0
S
(spike)
core   0: 0x0000000080004094 (0x10200073) sret
core   0: 1 0x0000000080004094 (0x10200073) c768_mstatus 0x0000000a000000a0 c15
(spike) priv 0
VS
(spike)
```

```
(spike) priv 0
VS
(spike)
core   0: 0x0000000080005020 (0x00000073) ecall
core   0: exception trap_virtual_supervisor_ecall, epc 0x0000000080005020
(spike) priv 0
S
(spike)
```

```
(spike) priv 0
S
(spike)
core   0: 0x000000008000401c (0x00000073) ecall
core   0: exception trap_supervisor_ecall, epc 0x000000008000401c
(spike) priv 0
M
(spike)
```

l): Confirm test exited successfully in M-mode with tohost=1

```
(spike)
core   0: >>>>  test_pass
core   0: 0x0000000080005044 (0x00100513) li      a0, 1
core   0: 1 0x0000000080005044 (0x00100513) x10 0x0000000000000001
(spike)
core   0: 0x0000000080005048 (0x00c0006f) j       pc + 0xc
core   0: 1 0x0000000080005048 (0x00c0006f)
(spike)
core   0: >>>>  write_tohost
core   0: 0x0000000080005054 (0xffffcf17) auipc   t5, 0xffffc
core   0: 1 0x0000000080005054 (0xffffcf17) x30 0x0000000080001054
(spike)
core   0: 0x0000000080005058 (0xfacf0f13) addi    t5, t5, -84
core   0: 1 0x0000000080005058 (0xfacf0f13) x30 0x0000000080001000
(spike)
core   0: 0x000000008000505c (0x00af3023) sd      a0, 0(t5)
core   0: 1 0x000000008000505c (0x00af3023) mem 0x0000000080001000 0x0000000000000001
BilalAli_Grand_Assessment$
```

Reference to Specification

RISC-V Privileged Architecture Specification

- **Section 8.2.1:** status register
- **Section 8.2.10:** hgatp register
- **Section 8.3:** Hypervisor Instructions
- **Section 8.5:** Two-Stage Address Translation
- **Section 8.5.3:** Memory-Management Fences